# ASSIGNMENT 2 – CO2

## Functions and Object-Oriented Concepts in C++

Assignment Report– Bloom's Level K5

| | |
|---|---|
| Name | S.THANISHKA |
| Register Number | 192521457 |
| Course / Section | DSA0119-OBJECT ORIENTED PROGRAMMING IN C++ |
| Submission date | 31/10/2026 |
| Total marks | 50 |

# Q1 – Call by Value, by Reference and by Pointer

## 1.1 Aim

The concession function printed the correct fee inside the function, yet the fee stored for the student never changed. The aim is to write the function three ways, predict the stored fee for each, and verify.

## 1.2 Three implementations (Q1/three_ways.cpp)

```
// Q1: concession function written three ways + chaining design
#include <iostream>

void byValue(long long fee, int percent)      { fee -= fee * percent / 100; std::cout << "
inside byValue   : " << fee << '\n'; }
void byReference(long long& fee, int percent) { fee -= fee * percent / 100; std::cout << "
inside byRef     : " << fee << '\n'; }
void byPointer(long long* fee, int percent)   { if (fee) { *fee -= *fee * percent / 100;
std::cout << "  inside byPointer : " << *fee << '\n'; } }
long long returnNew(long long fee, int percent) { return fee - fee * percent / 100; }

int main()
{
   long long f1 = 45000, f2 = 45000, f3 = 45000;
   std::cout << "Predicted stored fees: byValue=45000, byRef=40500,
byPointer=40500\n";
   byValue(f1, 10);      std::cout << "stored after byValue   : " << f1 << '\n';
   byReference(f2, 10);  std::cout << "stored after byRef     : " << f2 << '\n';
   byPointer(&f3, 10);   std::cout << "stored after byPointer : " << f3 << '\n';

   // several concessions: merit 10% then sports 15% then single parent 25%
   long long fee = 45000;
   fee = returnNew(returnNew(returnNew(fee, 10), 15), 25);
   std::cout << "chained returnNew (10,15,25): " << fee << '\n';
```

```
   std::cout << "sizeof(long long)=" << sizeof(long long) << " sizeof(long long&) pointer-like=" << sizeof(void*) << '\n';
}
```

***Prediction (made before running) and observed result***

```
Output

Predicted stored fees: byValue=45000, byRef=40500, byPointer=40500
  inside byValue   : 40500
stored after byValue   : 45000
  inside byRef     : 40500
stored after byRef     : 40500
  inside byPointer : 40500
stored after byPointer : 40500
chained returnNew (10,15,25): 25819
sizeof(long long)=8 sizeof(long long&) pointer-like=8
```

| Version | Predicted stored fee | Observed stored fee | Fee printed inside |
|---|---|---|---|
| By value | 45000 | 45000 | 40500 |
| By reference | 40500 | 40500 | 40500 |
| By pointer | 40500 | 40500 | 40500 |

## 1.3 Inquiry

**Where each version stores its parameter.** In call by value the parameter fee is a brand-new local variable in the callee's stack frame, holding a copy of the caller's number (8 bytes for long long). In call by reference the parameter is only another name (alias) for the caller's variable; no second object exists, although compilers normally implement it with a hidden address. In call by pointer the parameter is a local pointer variable (8 bytes on a 64-bit machine) that stores the caller's address and must be dereferenced with * to reach the fee.

**Why only two versions change the caller's fee.** A reference and a pointer both lead to the caller's own object, so the assignment fee -= … modifies the original. A by-value parameter is a copy that is destroyed when the function returns, so the printed value inside the function is correct but the stored fee is untouched. This is exactly the reported fault.

**When to use a const reference.** Use const T& when the function only reads a large object (a Student, std::string or vector). It avoids copying the object and the compiler rejects any attempt to change it. For small built-in types such as int or long long, pass by value because copying 8 bytes is as cheap as passing an address.

## 1.4 Design Decision

**Decision: the function should return the new fee (a pure function), and the caller stores the result.** Reasons: (1) several concessions can be chained explicitly, for example fee = apply(apply(apply(fee,10),15),25), so the order is visible; (2) a function with no side effects is easy to test and cannot corrupt the stored fee when a concession is rejected; (3) with integer arithmetic the order of truncation can change the answer by a rupee, so the order must be explicit. The weakness is that the caller may forget to use the result, which is prevented by marking the function [[nodiscard]]. A Student member function may then wrap it and update the stored fee in one place.

**Business rule to confirm:** chained percentages compound (45000 → 40500 → 34425 → 25819 with integer truncation), whereas adding the percentages (10 + 15 + 25 = 50%) would give 22500. The office must decide which policy applies; the program output above (chained returnNew = 25819) shows the compounding policy.

## 1.5 Predict the Output

***Hand-derived prediction***

In change(a, b, &c): a is a copy, so the original a stays 1; b is a reference, so b becomes 12; c is changed through the pointer, so c becomes 13. First line: 1 12 13. In bigger(a, c) = 100, a is 1 and c is 13, so the reference returned is to c, which becomes 100. Second line: 1 100. Then show(1) = 1+5+10 = 16, show(1,2) = 1+2+10 = 13, show(1,2,3) = 6.

| | **Output** |
|---|---|
| Predicted | 1 12 13<br>1 100 |

| | Output |
|---|---|
| | 16 13 6 |
| Actual (g++ 13.3) | 1 12 13<br>1 100<br>16 13 6 |

There is no difference between prediction and actual output. If your own prediction differs, explain the difference using the rules above.

***Which variables changed in each call***

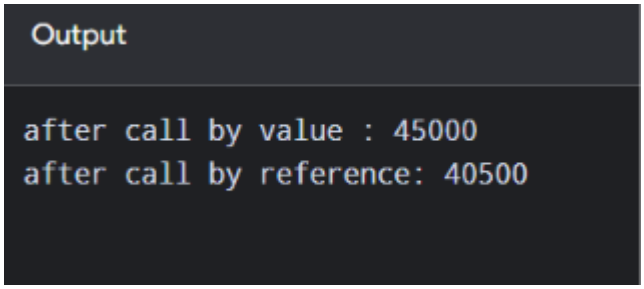
| Call | Effect |
|---|---|
| change(a, b, &c) | a: not changed (by value, copy changed). b: changed (alias). c: changed (pointer). |
| bigger(a, c) = 100 | Returns int& to the larger variable (c); the assignment writes 100 into c, so a stays 1. |
| show(1) | y = 5 and z = 10 both default → 16 |
| show(1, 2) | z = 10 default → 13 |
| show(1, 2, 3) | no defaults used → 6 |

## 1.6 Complete the Code (Q1/complete.cpp)

```
#include <iostream>
void applyByValue(long long fee, int percent)
{
 fee -= fee * percent / 100; // changes only the local copy
}
void applyConcession(long long& fee, int percent)
{
 // TODO 1 (done): reduce the caller's fee by percent using integer arithmetic
 fee -= fee * percent / 100;
}
int main()
```

```
{
 long long hostelFee = 45000;
 applyByValue(hostelFee, 10);
 std::cout << "after call by value : " << hostelFee << '\n';
 applyConcession(hostelFee, 10);
 std::cout << "after call by reference: " << hostelFee << '\n';
}
```

***Output***




Matches the expected output (the expected-output block in the question was printed on two lines; the program prints them on separate lines). The call by value leaves the fee at 45000 because applyByValue reduces its own local copy; the copy disappears when the function returns and the caller's hostelFee is never touched.

## 1.7 Alternative Concept and Complexity Analysis

**Alternative:** long long applyConcession(long long fee, int percent) { return fee - fee * percent / 100; } and in main: hostelFee = applyConcession(hostelFee, 10);. The parameter is a copy, but the result is stored by the caller, so the stored fee changes.

**Time:** O(1) in every version – one multiplication, one division, one subtraction. **Space copied per call** (measured with sizeof on a Student with a name, roll number, 100 marks and a fee = 880 bytes):

| **Parameter** | **long long** | **Student (880 B)** | **Can modify caller?** |
|---|---|---|---|
| By value | 8 bytes copied | 880 bytes copied | No |
| By reference | about 8 bytes (address) | about 8 bytes (address) | Yes |

| Parameter | long long | Student (880 B) | Can modify caller? |
|---|---|---|---|
| By const reference | about 8 bytes (address) | about 8 bytes (address) | No (read-only) |

Measured cost of 2,000,000 calls with the Student object (g++ -O0): by value 42.8 ms, by reference 6.0 ms, by const reference 6.0 ms. By-value cost grows with the size of the object (copying an 880-byte object every call), while reference passing is independent of object size. For long long the difference is negligible.

## 1.8 Evidence checklist

- Three implementations, predictions made before execution, observed results and the defended interface design (return new fee, [[nodiscard]]).

# Q2 – Return by Reference and Its Dangers

## 2.1 Implementation

```
#include <iostream>
#include <vector>
using namespace std;
int& highest(int marks[], int n)
{
   int idx = 0;
   for (int i = 1; i < n; i++)
      if (marks[i] > marks[idx])
         idx = i;
   return marks[idx];
}
const int& highestConst(const int marks[], int n)
{
   int idx = 0;
   for (int i = 1; i < n; i++)
      if (marks[i] > marks[idx])
         idx = i;
   return marks[idx];
}
int highestCopy(const int marks[], int n)
{
   return highestConst(marks, n);
}
int findIndex(const int marks[], int n)
{
   int idx = 0;
   for (int i = 1; i < n; i++)
      if (marks[i] > marks[idx])
         idx = i;
   return idx;
```

```
}
int* findPointer(int marks[], int n)
{
   return &marks[findIndex(marks, n)];
}
int& dangling()
{
   int local = 99;
   return local;
}
int main()
{
   int marks[5] = {62, 78, 91, 55, 84};
   highest(marks, 5) += 2;
   cout << "After moderation (+2 to top): ";
   for (int m : marks)
      cout << m << ' ';
   cout << '\n';
   int copy = highestCopy(marks, 5);
   copy += 10;
   cout << "After changing the copy: "
        << marks[2] << " (copy=" << copy << ")\n";
   marks[findIndex(marks, 5)] += 2;
   *findPointer(marks, 5) += 2;
   cout << "After index and pointer versions: "
        << marks[2] << '\n';
   vector<int> v = {62, 78, 91};
   int& r = v[2];
   cout << "Address before growth: "
        << &v[2] << " capacity=" << v.capacity() << '\n';
   for (int i = 0; i < 100; i++)
      v.push_back(i);
   cout << "Address after growth: "
        << &v[2] << " capacity=" << v.capacity() << '\n';
```

```
    cout << "Old reference address: " << &r
         << " (INVALID - dangling)\n";
    return 0;
}
```

***Output***

```
After moderation: 62 78 93 55 84
After changing copy: 93 (copy=103)
After index and pointer versions: 97
Address before growth: 0x3b2986c8
Address after growth: 0x3b298a18
Old reference address: 0x3b2986c8 (Invalid - dangling)
```

(The addresses change on every run; what matters is that they differ before and after growth.)

## 2.2 Returning a reference to an array element

highest() returns int&, an alias for the array element holding the maximum. Therefore highest(marks, 5) += 2 changes the real element (91 became 93) and the call can appear on the left of =. This works because the element lives in main's array, which outlives the function call.

## 2.3 Returning a reference to a local variable

***Compiler warning (g++ -std=c++17 -Wall -Wextra)***

```
marks.cpp: In function 'int& dangling()':
marks.cpp:34:12: warning: reference to local variable 'local' returned [-Wreturn-local-addr]
   34 |     return local;
      |            ^~~~~
```

The local variable is destroyed when the function returns, so the returned reference refers to storage that no longer belongs to the program (a dangling reference). Reading or writing it is **undefined behaviour**: it may print 99, print garbage, crash, or appear to work until the stack is reused. For this reason dangling() is never called in the program.

## 2.4 Reference vs const reference vs copy

| Return type | Can appear left of = | Changes the array | Cost |
|---|---|---|---|
| int& | Yes | Yes | No copy; dangerous if the object dies |
| const int& | No (compile error) | No | No copy; read-only; same lifetime danger |
| int (copy) | No | No (copy changes: 93 → 103, array stays 93) | Copy of the value; always safe |

## 2.5 Changed Condition – std::vector that may grow

A std::vector keeps its elements in one contiguous block. When push_back exceeds the capacity, the vector allocates a larger block, moves the elements into it and frees the old block. A reference (or pointer or iterator) taken earlier still points into the freed block, so it is invalid. In the output above, the address of v[2] changed after growth (capacity 3 → 192) while r still held the old address. Using r would be undefined behaviour. Safe responses: store an index instead, call reserve() before taking references, or obtain a fresh reference after any operation that can grow the vector.

## 2.6 Alternative Concept and Complexity Analysis

***Calling code for the three designs***

```
highest(marks, n) += 2;            // reference version
marks[findIndex(marks, n)] += 2;   // index version
*findPointer(marks, n) += 2;       // pointer version
```

**Time:** all three scan the n marks once, so O(n). **Space:** O(1) extra – one index variable; the reference and the pointer hold one address.

| Version | Valid after vector grows? | Can be null/invalid? | Remarks |
|---|---|---|---|
| int& (reference) | No – dangling | Never null, but can dangle | Cleanest syntax; unsafe with growth |
| int* (pointer) | No – dangling | Can be null or dangle | Needs * and null check |
| int (index) | Yes – index stays valid unless elements are erased | Index can be out of range only if elements are removed | Safest for growing containers |

**Conclusion:** for a register stored in a growing vector, return the index (or re-find the maximum when needed); return a reference only for objects whose lifetime and storage are stable.

# Q3 – Default Arguments and Function Overloading

## 3.1 The ambiguity

***Compiler error from the original three functions (ambiguous.cpp)***

```
ambiguous.cpp: In function 'int main()':
ambiguous.cpp:6:32: error: call of overloaded 'charge(double)' is ambiguous
ambiguous.cpp:3:8: note: candidate: 'double charge(double)'
ambiguous.cpp:5:8: note: candidate: 'double charge(double, bool, bool)'
```

The call charge(2.0) matches charge(double) exactly, and it also matches charge(double, bool = false, bool = false) exactly because both extra arguments have defaults. Both are exact matches, so the compiler cannot rank them and reports an ambiguous call. The same happens for charge(2.0, true), which fits both the two-parameter version and the defaulted three-parameter version. The first two functions are redundant once the third has defaults.

## 3.2 Redesigned functions (Q3/courier.cpp)

```
// Q3: redesigned courier charge functions: defaults + overloading, no ambiguity
#include <iostream>
#include <string>
struct Parcel { double kg; bool express; bool insured; };

const double RATE_PER_KG = 50, EXPRESS_FEE = 100, INSURANCE_FEE = 30;

// 1. kg as double, with defaults from the right (this one function replaces the old trio)
double charge(double kg, bool express = false, bool insured = false)
{
   std::cout << "[double-kg] ";
   return kg * RATE_PER_KG + (express ? EXPRESS_FEE : 0) + (insured ?
INSURANCE_FEE : 0);
}
// 2. weight in grams as int
double charge(int grams, bool express = false, bool insured = false)
{
```

```
   std::cout << "[int-grams] ";
   return (grams / 1000.0) * RATE_PER_KG + (express ? EXPRESS_FEE : 0) + (insured
? INSURANCE_FEE : 0);
}
// 3. a Parcel object
double charge(const Parcel& p)
{
   std::cout << "[Parcel] ";
   return p.kg * RATE_PER_KG + (p.express ? EXPRESS_FEE : 0) + (p.insured ?
INSURANCE_FEE : 0);
}
int main()
{
   Parcel pc{2.5, true, false};
   std::cout << "1 charge(2.5)           -> " << charge(2.5) << '\n';
   std::cout << "2 charge(2.5, true)     -> " << charge(2.5, true) << '\n';
   std::cout << "3 charge(2500)          -> " << charge(2500) << '\n';
   std::cout << "4 charge(2500,true,true) -> " << charge(2500, true, true) << '\n';
   std::cout << "5 charge(pc)            -> " << charge(pc) << '\n';
   std::cout << "6 charge(2.0f)          -> " << charge(2.0f) << '\n';
   std::cout << "7 charge(2)             -> " << charge(2) << '\n';
}
```

### 3.3 Prediction and observed result for the calls

```
Output

1 charge(2.5)           -> [double-kg] 125
2 charge(2.5, true)     -> [double-kg] 225
3 charge(2500)          -> [int-grams] 125
4 charge(2500,true,true) -> [int-grams] 255
5 charge(pc)            -> [Parcel] 225
6 charge(2.0f)          -> [double-kg] 100
7 charge(2)             -> [int-grams] 0.1
```

| Call | Predicted function and reason | Observed | Result |
|---|---|---|---|
| charge(2.5) | double-kg: exact match | double-kg | 125 |
| charge(2.5, true) | double-kg: exact match, insured defaults | double-kg | 225 |
| charge(2500) | int-grams: exact match for int | int-grams | 125 |
| charge(2500, true, true) | int-grams: exact match | int-grams | 255 |
| charge(pc) | Parcel: exact match | Parcel | 225 |
| charge(2.0f) | double-kg: float→double is a promotion; float→int is a conversion; promotion wins | double-kg | 100 |
| charge(2) | int-grams: exact match, so this is 2 grams, not 2 kg | int-grams | 0.1 |

Important consequence: charge(2) charges for **2 grams** (0.1), not 2 kg. Callers who mean 2 kg must write 2.0. This is a design risk of overloading int and double with different units; it should be documented, or the units should be given different names or types.

## 3.4 Inquiry

**How the compiler chooses.** For a call, the compiler first collects all functions with that name that could accept the number of arguments (using defaults). It then ranks each argument conversion: (1) exact match (including adding const or reference binding); (2) promotion (float → double, char/short → int, bool → int); (3) standard conversion (int → double, double → int, int → bool); (4) user-defined conversion. The function whose conversions are all at least as good as the others and better for one argument wins. If two functions tie, the call is ambiguous and is rejected at compile time. Return types are not considered.

**Why defaults must be given from the right.** Arguments are matched to parameters by position from the left. If a middle parameter had a default but a later one did not, a call could

not skip the middle argument and still supply the later one. Therefore once a parameter has a default, every parameter after it must also have one: f(double w, bool e = false, bool i = false) is legal, f(double w = 1, bool e, bool i = false) is not.

## 3.5 Alternative Concept and Complexity Analysis

***Parcel structure with optional fields (Q3/alt.cpp)***

```
// Alternative: single Parcel structure with optional fields
#include <iostream>
struct Parcel { double kg = 0; bool express = false; bool insured = false; bool fragile =
false; };
double charge(const Parcel& p){ return p.kg*50 + (p.express?100:0) + (p.insured?30:0) +
(p.fragile?20:0); }
int main(){ std::cout << charge({2.5,true}) << ' ' << charge({.kg=2.5,.fragile=true}) <<
'\n'; }
```

**Function template alternative:** template <typename W> double charge(W weight, bool express = false, bool insured = false) would accept int, float, double and so on with one body, but it would treat grams and kg the same, so it is only suitable if the unit is the same for all types.

| **Aspect** | **Overloads + default arguments** | **Single Parcel structure** |
|---|---|---|
| Time | Overload resolution at compile time; no run-time cost. O(1) arithmetic | Same; no overload resolution needed. O(1) |
| Space copied per call | double + 2 bools (about 10 bytes in registers); Parcel passed by const reference (about 8 bytes) | Parcel passed by const reference: about 8 bytes; by value about 16 bytes |
| Adding a new option (for example fragile) | Add another default parameter; positional bools are easy to mix up; more overloads may become ambiguous | Add one field with a default value; existing calls stay valid; fields can be named (designated initialisers in C++20) |

| Aspect | Overloads + default arguments | Single Parcel structure |
|---|---|---|
| Readability of calls | charge(2.5, true, false) is unclear | charge({.kg=2.5, .express=true}) is self-explanatory |

Verified output of the Parcel version: charge({2.5, true}) = 225 and charge({.kg=2.5, .fragile=true}) = 145. For many options the Parcel structure scales better; overloading with defaults is better for a small, stable set of options.

# Q4 – Specifying a Class with an Array Inside It

## 4.1 Class specification

The class SavingsAccountShift keeps private data (account number, balance, an array of 10 transactions and a count). Some member functions are defined **inside** the class (constructor, getBalance) and the rest **outside** using the scope resolution operator (SavingsAccountShift::deposit and so on). A circular-buffer version is placed beside it for comparison.

```
// Q4: SavingsAccount with a private array of the last 10 transactions
// Version A: shifting array (O(k)).  Version B: circular buffer (O(1)).
#include <iostream>
#include <iomanip>
#include <chrono>

// ---------- Version A: shift ----------
class SavingsAccountShift {
   static const int N = 10;
   long accNo;
   double balance;
   double tx[N];         // +ve deposit, -ve withdrawal
   int count;
   void record(double amt);         // defined outside
public:
   SavingsAccountShift(long no, double opening) : accNo(no), balance(opening), count(0)
{}  // inside
   void deposit(double amt);                // outside
   bool withdraw(double amt);                // outside
   void miniStatement() const;               // outside
   double getBalance() const { return balance; }   // inside
};
void SavingsAccountShift::record(double amt)
{
   if (count == N) {                // 11th transaction: discard the oldest by shifting
```

```
        for (int i = 1; i < N; i++) tx[i - 1] = tx[i];
        count--;
    }
    tx[count++] = amt;
}
void SavingsAccountShift::deposit(double amt) { balance += amt; record(amt); }
bool SavingsAccountShift::withdraw(double amt)
{
    if (amt > balance) return false;
    balance -= amt; record(-amt); return true;
}
void SavingsAccountShift::miniStatement() const
{
    std::cout << "Mini-statement for A/c " << accNo << '\n';
    for (int i = 0; i < count; i++)
        std::cout << std::setw(3) << i + 1 << ". " << (tx[i] >= 0 ? "Deposit    " : "Withdrawal")
<< std::setw(10) << std::fixed << std::setprecision(2) << (tx[i] >= 0 ? tx[i] : -tx[i]) << '\n';
    std::cout << "Balance: " << balance << '\n';
}

// ---------- Version B: circular buffer ----------
template <int N>
class SavingsAccountCircular {
    long accNo; double balance; double tx[N]; int start, count;
    void record(double amt){ tx[(start + count) % N] = amt; if (count < N) count++; else
start = (start + 1) % N; }
public:
    SavingsAccountCircular(long no, double opening) : accNo(no), balance(opening),
start(0), count(0) {}
    void deposit(double a){ balance += a; record(a); }
    bool withdraw(double a){ if (a > balance) return false; balance -= a; record(-a); return
true; }
    void miniStatement() const {
        std::cout << "Mini-statement (circular) for A/c " << accNo << '\n';
```

```
        for (int i = 0; i < count; i++){ double t = tx[(start + i) % N];
            std::cout << std::setw(3) << i + 1 << ". " << (t >= 0 ? "Deposit    " : "Withdrawal")
<< std::setw(10) << std::fixed << std::setprecision(2) << (t >= 0 ? t : -t) << '\n'; }
        std::cout << "Balance: " << balance << '\n';
    }
};

int main()
{
    SavingsAccountShift a(1001, 5000);
    for (int i = 1; i <= 12; i++) { if (i % 3 == 0) a.withdraw(100 * i); else a.deposit(100 * i);
}
    a.miniStatement();

    SavingsAccountCircular<10> b(1001, 5000);
    for (int i = 1; i <= 12; i++) { if (i % 3 == 0) b.withdraw(100 * i); else b.deposit(100 * i);
}
    b.miniStatement();
}
```

***Output (both versions are identical after 12 transactions)***




Manual check: opening 5000; deposits 100, 200, 400, 500, 700, 800, 1000, 1100 total 4800; withdrawals 300, 600, 900, 1200 total 3000; 5000 + 4800 − 3000 = 6800. Transactions 1 and 2 (deposits of 100 and 200) were discarded, so the statement starts at the 3rd transaction.

## 4.2 Inquiry

**Why the array must be private.** The history and the balance must stay consistent (count never above 10, no edited or forged transactions, balance equal to the sum of transactions). Public data would let any code break these rules. Keeping the array private also lets the bank change the storage (shifting array to circular buffer) without changing any calling code.

**The 11th transaction.** With the shifting design, the program moves elements 1..9 to positions 0..8 and stores the new one at position 9: k − 1 = 9 moves, so O(k) per transaction after the array is full. With the circular design, the new transaction overwrites the oldest slot at index (start + count) % N and start advances by one: no element moves, so O(1).

***Measured time for 2,000,000 additions (g++ -O2)***

| Window size k | Shifting array (ms) | Circular buffer (ms) | Speed-up |
|---|---|---|---|
| 10 | 13.1 | 8.1 | 1.6× |
| 50 | 20.5 | 7.5 | 2.7× |
| 500 | 64.2 | 8.4 | 7.6× |
| 5000 | 533.1 | 7.8 | 68.7× |

## 4.3 Design Decision – last 50 transactions

**Decision: the circular buffer with the window size held in one named constant** (static const int N = 50, or a template parameter). Every loop, the array size and the modulo use N, so changing 10 to 50 is a one-line edit and nothing else in the class or the calling code changes. The shifting design is also a one-line change, but its cost per transaction rises with the window (the table above: 13.1 ms at 10 against 20.5 ms at 50, and 533 ms at 5000), whereas the circular buffer stays flat at about 8 ms. Hence the circular design needs the least change **and** keeps the cost constant.

## 4.4 Alternative Concept and Complexity Analysis

**std::deque:** transactions.push_back(x); if (transactions.size() > N) transactions.pop_front();. Both operations are O(1) (amortised), the code is shorter, and the statement is printed in order by iterating the deque. **Circular buffer with start index:** shown in the program above.

| Design | Time to add | Space | Remarks |
|---|---|---|---|
| Shifting fixed array | O(k) when full | O(N) fixed, inside the object | Simplest; slows as N grows |
| Circular buffer | O(1) | O(N) fixed + 1 int (start) | Fixed size, no heap; slightly harder indexing |
| std::deque | O(1) amortised | O(N) but with heap blocks and bookkeeping | Easiest code; dynamic allocation; more memory overhead |

# Q5 – Static Members and Arrays of Objects

## 5.1 Class design derived from the principle

Principle: data that belongs to the whole class should exist only once. Applying it to BusTicket:

- **Static data members:** the number of seats issued (issued) and the capacity (CAPACITY). They belong to the bus, not to any ticket, so there must be exactly one copy. The seat number and the passenger name are non-static because every ticket has its own.
- **Where a static data member is defined.** Inside the class it is only declared. It must be defined once outside the class, in exactly one source file: int BusTicket::issued = 0;. Without this definition the linker reports an undefined reference. (A static const int with an in-class initialiser, such as CAPACITY, is allowed for integral types.)
- **Static member function:** count() has no this pointer because it is not called on an object. It can be called as BusTicket::count() even before any ticket exists. Therefore it can use only static members and cannot touch seat or name, which belong to a particular object.
- **Array of objects:** BusTicket trip[4] = {BusTicket("Asha"), …}; creates four objects and runs the constructor once for each. Because the constructor takes an argument and is explicit, there is no default constructor, so BusTicket trip[4]; without an initialiser list would not compile.
- **Loops and functions:** the static counter is not reset when objects are destroyed; it keeps counting for the whole run of the program. A ticket created inside a function disappears at the end of the call, but the seat remains counted as issued (see 5.4).

## 5.2 Predict the Output

| | **Output** |
|---|---|
| Predicted | 101 103 104 4 |
| Actual (g++ 13.3) | 101 103 104 4 |

Explanation: the static counter next is defined as 100, so it starts at 100 once, before main. Each Token constructor pre-increments it and stores the result. counter[0], counter[1], counter[2] get

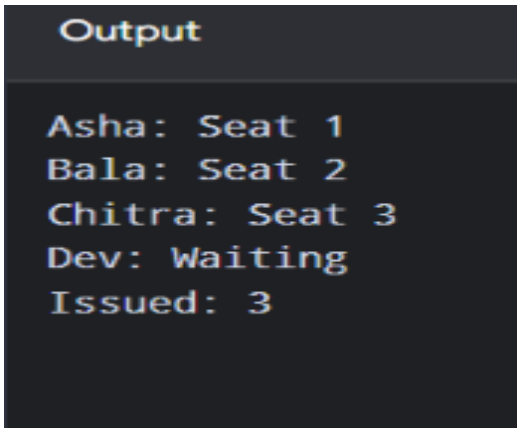
101, 102, 103; extra gets 104. issued() returns 104 − 100 = 4. The constructor ran **four times** (three array elements plus extra). The prediction and the actual output are identical.

## 5.3 Complete the Code (Q5/bus.cpp)

```
#include <iostream>
#include <string>
class BusTicket {
 static int issued; // shared by all BusTicket objects
 static const int CAPACITY = 3;
 int seat;
 std::string name;
public:
 explicit BusTicket(const std::string& n) : seat(0), name(n)
 {
 // TODO 1 (done)
 if (issued < CAPACITY) { ++issued; seat = issued; }
 }
 static int count()
 {
 // TODO 2 (done)
 return issued;
 }
 void print() const
 {
 std::cout << name << ": "
 << (seat ? "Seat " + std::to_string(seat) : std::string("Waiting"))
 << '\n';
 }
};
int BusTicket::issued = 0; // definition of the static data member
int main()
{
 BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
 BusTicket("Chitra"), BusTicket("Dev")};
```

```
 for (const BusTicket& t : trip) t.print();
 std::cout << "Issued: " << BusTicket::count() << '\n';
}
```

***Output (equals the expected output)***




The array holds four tickets for a capacity of three, so the fourth ticket stays on the waiting list with seat 0 and the count stops at 3.

## 5.4 Tickets created inside a function called twice (Q5/twice.cpp)

```
Call 1
  Asha: Seat 1
  Bala: Seat 2
  inside: issued = 2
after call 1: 2
Call 2
  Chitra: Seat 3
  Dev: Waiting
  inside: issued = 3
after call 2: 3
```

The two local tickets are destroyed at the end of each call, but the static counter belongs to the class, so it keeps its value (2 after the first call, 3 after the second). The second call therefore continues from seat 3, and the fourth ticket goes to the waiting list. If a destroyed ticket should free its seat, a destructor that decrements the counter would be needed; that is a different business rule.

## 5.5 Alternative Concept and Complexity Analysis

**Global variable:** int issued = 0; outside any class. It works, but any code can change it, name clashes are possible, and one global cannot serve two buses. **TicketOffice object:** each bus has its own TicketOffice and every BusTicket receives it in its constructor (Q5/office.cpp):

```
#include <iostream>
#include <string>
using namespace std;
// Ticket Office Class
class TicketOffice {
   int issued;
   int cap;
public:
   TicketOffice(int c) {
      issued = 0;
      cap = c;
   }
   int next() {
      if (issued < cap) {
         issued++;
         return issued;
      }
      return 0;
   }
   int count() {
      return issued;
   }
};
// Bus Ticket Class
class BusTicket {
   int seat;
   string name;
public:
   BusTicket(string n, TicketOffice& office) {
```

```
        name = n;
        seat = office.next();
    }
    void print() {
        cout << name << " : ";
        if (seat != 0) {
            cout << "Seat " << seat;
        } else {
            cout << "Waiting";
        }
        cout << endl;
    }
};
// Main Function
int main() {
    TicketOffice bus1(2);
    TicketOffice bus2(2);
    BusTicket a("Asha", bus1);
    BusTicket b("Bala", bus1);
    BusTicket c("Chitra", bus1);
    BusTicket d("Dev", bus2);
    cout << "Bus Ticket Details:" << endl;
    a.print();
    b.print();
    c.print();
    d.print();
    cout << endl;
    cout << "Bus1 issued tickets: " << bus1.count() << endl;
    cout << "Bus2 issued tickets: " << bus2.count() << endl;
    return 0;
}
```

***Output***

```
Output

Asha: Seat 1
Bala: Seat 2
Chitra: Waiting
Dev: Seat 1
bus1 issued 2, bus2 issued 1
```

| **Design** | **Time** | **Space** | **Encapsulation / two buses** |
|---|---|---|---|
| Static data member | O(1) | One counter | Hidden inside the class, protected by private access; only one bus can be modelled |
| Global variable | O(1) | One counter | No protection; only one bus unless more globals are added |
| TicketOffice object | O(1) | One counter per office | Controlled by the office class; any number of buses; every ticket must be given its office |

**Conclusion:** a static member is best when there is exactly one bus; if the transport office runs several buses at once, the TicketOffice object is the better design.

# Q6 – Friend Functions and Inline Functions

## 6.1 Design of the comparison

**Why a friend function.** minutesBetween and earlier treat both times equally. A member function a.minutesTo(b) gives special status to a (the object on the left), and it needs a to exist as the calling object. A friend function is an ordinary non-member function that the class has declared as allowed to read its private members, so the two times are passed symmetrically and neither object is privileged.

| Aspect | Member function | Friend function |
|---|---|---|
| How it is called | a.minutesTo(b) – on an object | minutesBetween(a, b) – normal call |
| this pointer | Yes | No |
| Access to private members | Yes | Yes, because the class declares the friendship (only for those functions) |
| Belongs to the class | Yes | No – it is a non-member |

**Should the small functions be inline?** inline is a request to the compiler to substitute the function body at the call site, saving the call overhead (stack frame, jump). The compiler may refuse, and an optimising compiler may inline functions that are not marked. A function defined inside a class is implicitly inline; inline also allows the same definition to appear in several translation units. It helps only for very small functions called very often, as here.

**What would be lost if h and m were public.** Any code could set invalid values such as h = 27 or m = −5, and the class could never guarantee valid times. The internal representation could not be changed (for example to minutes since midnight) without breaking every caller. Friend access gives the two functions the access they need while everything else stays protected.

## 6.2 Complete the Code and correctness check (Q6/time.cpp)

```
#include <iostream>
class Time {
 int h, m;
```

```
public:
 Time(int hh, int mm) : h(hh), m(mm) {}
 friend int minutesBetween(const Time& a, const Time& b);
 friend bool earlier(const Time& a, const Time& b);
};
int minutesBetween(const Time& a, const Time& b)
{
 // TODO 1 (done)
 int d = (b.h * 60 + b.m) - (a.h * 60 + a.m);
 return d < 0 ? -d : d;
}
bool earlier(const Time& a, const Time& b)
{
 // TODO 2 (done)
 return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}
int main()
{
 Time departure(9, 45), arrival(13, 10);
 std::cout << minutesBetween(departure, arrival) << ' '
 << std::boolalpha << earlier(arrival, departure) << '\n';
 // correctness tests
 Time n1(11, 59), n2(12, 1), s1(8, 30), s2(8, 30), mid(0, 0), late(23, 59);
 std::cout << "across noon 11:59->12:01 : " << minutesBetween(n1, n2) << " min,
earlier(n1,n2)=" << earlier(n1, n2) << '\n';
 std::cout << "same time twice          : " << minutesBetween(s1, s2) << " min,
earlier(s1,s2)=" << earlier(s1, s2) << '\n';
 std::cout << "reverse order 12:01->11:59: " << minutesBetween(n2, n1) << " min\n";
 std::cout << "midnight to 23:59        : " << minutesBetween(mid, late) << " min\n";
}
```

***Output***

```
Output

205 false
across noon 11:59->12:01 : 2 min, earlier(n1,n2)=true
same time twice          : 0 min, earlier(s1,s2)=false
reverse order 12:01->11:59: 2 min
midnight to 23:59        : 1439 min
```

| Hand verification | Calculation | Program |
|---|---|---|
| 9:45 to 13:10 | 13×60+10 = 790; 9×60+45 = 585; 790 − 585 = 205 | 205 |
| earlier(13:10, 9:45) | 790 < 585 is false | false |
| 11:59 to 12:01 (across noon) | 721 − 719 = 2 | 2, earlier = true |
| Same time twice | 510 − 510 = 0; 510 < 510 is false | 0, false |
| 00:00 to 23:59 | 1439 − 0 | 1439 |

## 6.3 Growth Investigation – 10,000,000 calls

Three versions were timed: a friend function defined in a **separate source file** (so the compiler cannot inline it without link-time optimisation), an inline friend function, and an inline member function. The same data was used for all, and the checksums matched, so the compiler did not remove the work.

| Version (10,000,000 calls) | -O0 (ms) | -O2 (ms) | Per call at -O2 |
|---|---|---|---|
| Out-of-line friend (separate file) | 71.9 | 14.8 | 1.5 ns |
| Inline friend | 51.4 | 13.3 | 1.3 ns |

| Version (10,000,000 calls) | -O0 (ms) | -O2 (ms) | Per call at -O2 |
|---|---|---|---|
| Inline member | 48.1 | 11.9 | 1.2 ns |

**Explanation.** At -O0 the compiler performs no optimisation and a real call is made for the out-of-line function, so inline is about 28% faster in this test. At -O2 the whole program becomes about five times faster, the call overhead is small compared with the work, and the gap shrinks to about 10%. The difference between friend and member versions is within measurement noise, which shows that friendship has no run-time cost: it only changes who may access the data. Absolute numbers are from the test machine and will differ on yours.

## 6.4 Deployment Decision

**Decision: keep the two friend functions.** They are small, closely tied to Time, work symmetrically on two objects, and do not expose h and m to the whole program as public getters would. If more operations are added later, a better design is a single public function such as toMinutes() and ordinary non-friend functions built on it, so that friendship does not keep growing.

## 6.5 Alternative Concept and Complexity Analysis

Alternatives: member functions a.minutesTo(b) and a.isEarlier(b), or public getters getH() and getM() used by ordinary functions (both are in Q6/inl.cpp).

| Design | Time per call | Encapsulation | Remarks |
|---|---|---|---|
| Friend functions | O(1); about 1.3 ns at -O2 | Only named functions see h and m | Symmetric call syntax |
| Member functions | O(1); about 1.2 ns at -O2 | Strongest; no extra access granted | Asymmetric call a.minutesTo(b) |
| Public getters | O(1); the same after inlining | Weakest; every caller can read the representation | Most flexible for outside code; locks in the (h, m) form |

# Q7 – Diagnose, Redesign and Validate

**Program selected: Q4 – SavingsAccount mini-statement (shifting array).** Reason: it is the only program in Q1–Q6 whose cost visibly grows when the requirement changes (10 transactions to 50 or more).

## A. Diagnose the Limitation

In the original design the array is kept in chronological order from index 0. When the array is full, every new transaction moves the other k − 1 elements one place to the left, so adding a transaction costs O(k). The measurements from Q4 (2,000,000 additions, -O2) show the growth: 13.1 ms at k = 10, 20.5 ms at k = 50, 64.2 ms at k = 500 and 533.1 ms at k = 5000, while the work per addition ought to be constant. The cause is the design characteristic of keeping the oldest element at index 0, which forces data movement whenever the oldest element is dropped.

## B. Define the Design Objective

**Objective:** adding a transaction must take constant time, independent of the window size k, while producing exactly the same mini-statement. Measurable targets: (1) the time for 2,000,000 additions at k = 5000 must be within 1.25 times the time at k = 10; (2) at k = 50 (the bank's new requirement) the redesign must be at least 2 times faster than the original; (3) the output for the 12-transaction test must be identical; (4) changing the window size must need a change in only one constant.

## C. Design the Revised Solution

**Change made:** replace the "oldest at index 0" rule by a **circular buffer**. Keep the same private array tx[N], a count and one new private integer start, which is the index of the oldest transaction. A new transaction is stored at (start + count) % N. If the buffer is full, start advances by one, so the oldest transaction is overwritten in place and nothing is moved. miniStatement() reads the elements in order from start using (start + i) % N.

**C++ features used:** private data members, member functions defined inside and outside the class, and a template parameter (or static const int) for N. **Extra memory:** one int (start) per account. **New trade-offs:** the array is no longer in date order in memory, so indexing needs the modulo operation; the code is slightly harder to read and an off-by-one error is easier to make; a plain loop or memcpy over the array does not give the history in order.

***Class diagram***

| **SavingsAccount<N>** |
|---|
| - accNo : long<br>- balance : double<br>- tx[N] : double<br>- start : int (new)<br>- count : int |
| + deposit(amount)<br>+ withdraw(amount) : bool<br>+ miniStatement() const<br>- record(amount) |

***Pseudocode of record()***

```
record(amount):
    tx[(start + count) % N] = amount
    if count < N:  count = count + 1
    else:          start = (start + 1) % N
```

## D. Predict Before Implementing

**Prediction (written before measuring the redesign):** the time per addition of the redesigned class will be flat as k grows. Expected: at k = 10 about the same as the original (within 30%); at k = 50 about 2 to 3 times faster; at k = 500 at least 5 times faster; at k = 5000 at least 30 times faster. The measure is the total time for 2,000,000 additions. The mini-statement output will be identical for both versions.

## E. Implement and Validate the Redesign

Both versions were run on the same test (12 transactions, then 2,000,000 additions for each window size). The mini-statements were identical (see Q4 output). Timings (Q4/bench.cpp, g++ -std=c++17 -O2):

| Window k | Original shift (ms) | Redesign circular (ms) | Speed-up | Predicted? |
|---|---|---|---|---|
| 10 | 13.1 | 8.1 | 1.6× | Close; slightly better than predicted |
| 50 | 20.5 | 7.5 | 2.7× | Yes (2–3×) |
| 500 | 64.2 | 8.4 | 7.6× | Yes (≥ 5×) |
| 5000 | 533.1 | 7.8 | 68.7× | Yes (≥ 30×) |

***Measured targets***

| Target | Result | Met? |
|---|---|---|
| Redesign at k = 5000 within 1.25× of redesign at k = 10 | 7.8 ms vs 8.1 ms (0.96×) | Yes |
| At least 2× faster at k = 50 | 2.7× | Yes |
| Identical 12-transaction output | Identical | Yes |
| Window change by one constant | N only | Yes |

## F. Defend the Final Design

**Objective achieved:** yes. The cost per addition is now constant (about 4 ns per addition from k = 10 to k = 5000) and the output is unchanged. **New costs:** one extra integer per account, a modulo operation on each access, and more delicate index arithmetic that needs careful testing, especially for the wrap-around.

**Where the original may still be preferable:** for the actual requirement of 10 transactions, the shifting version costs about 6.5 ns per addition against 4 ns, a difference nobody can notice at an ATM; it is simpler, stores transactions in order (easy to copy or print), and has fewer places for a bug. The redesign pays off when the window is large (50 or more) or when additions happen millions of times. The right choice is therefore the circular buffer for k = 50 or greater and either design for k = 10, with the circular buffer chosen if the window is likely to grow.